

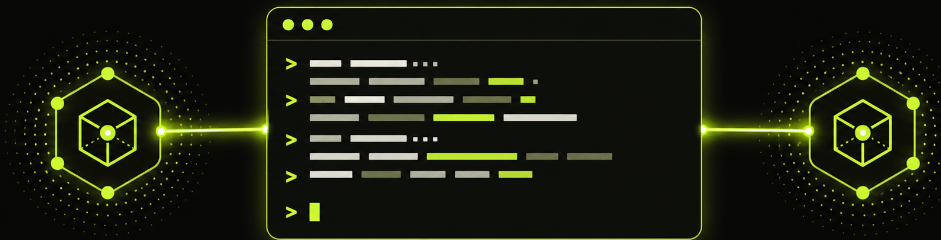
p2p-netcat

Netcat, адресованный PeerId

Один криптографический адрес – для терминала, TCP, UDP и прокси через NAT.

p2p-netcat web

Терминал между двумя узлами. Прямо из браузера.



IDENTITY

PeerId вместо IP

ROUTE

Первый проверенный путь

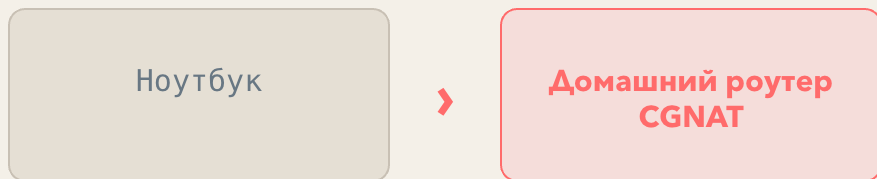
SESSION

TCP · UDP · PTY · SOCKS

У домашнего сервера нет публичного адреса – но вам всё равно нужен SSH.

БЕЗ P2P-NETCAT

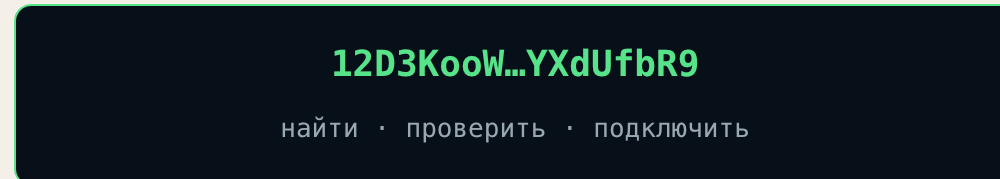
Сетевая граница вам не принадлежит



Проброс порта недоступен. Dynamic DNS не помогает. Размещённый VPN добавляет ещё один control plane.

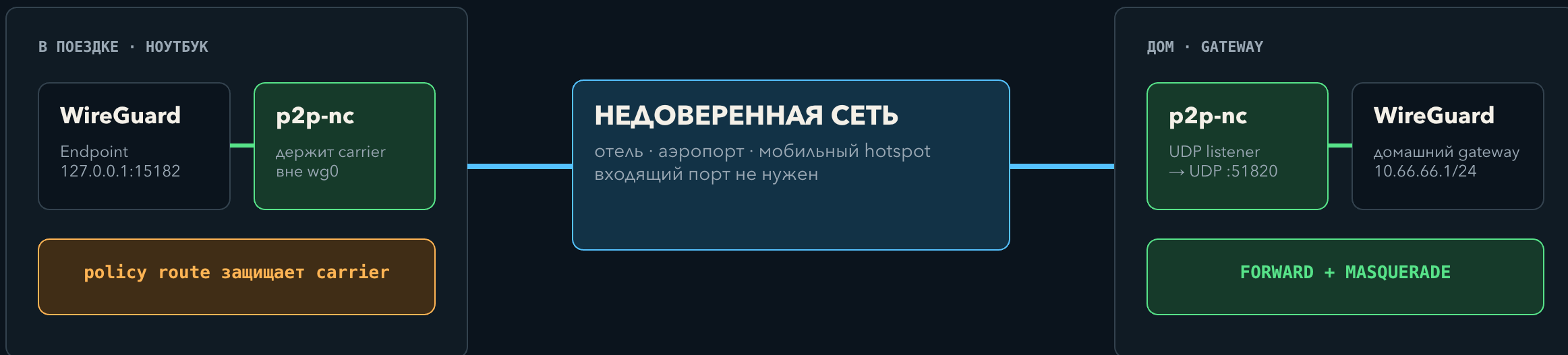
С P2P-NETCAT

У сервера остаётся одна identity



Запустите listener дома. Из любой сети наберите его PeerId и пробросьте локальный порт 2222 к SSH.

Полный интернет-туннель должен работать, даже если gateway находится за NAT.



ОДИН МАРШРУТ

запустить carrier



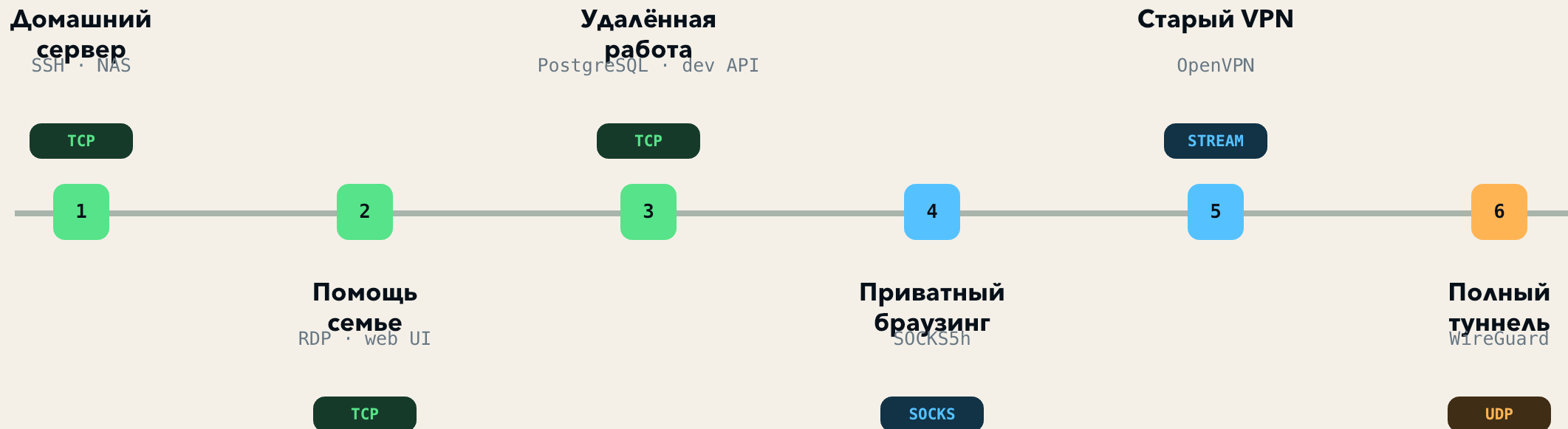
wg-quick up wg0



публичный IP = домашний

P2P-carrier сначала достигает домашнего пира; затем WireGuard направляет через него весь интернет-трафик.

Потребность повторяется: достичь закрытого сервиса, не владея сетевой границей.

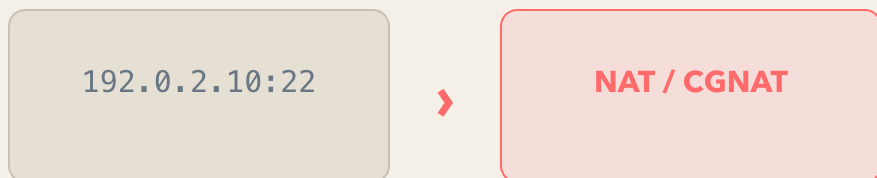


Один стабильный PeerId заменяет IP-адреса сетей, пробросы портов и временные VPN control plane.

PeerId решает identity. Маршрут всё равно нужно найти.

КЛАССИЧЕСКИЙ FORWARDING

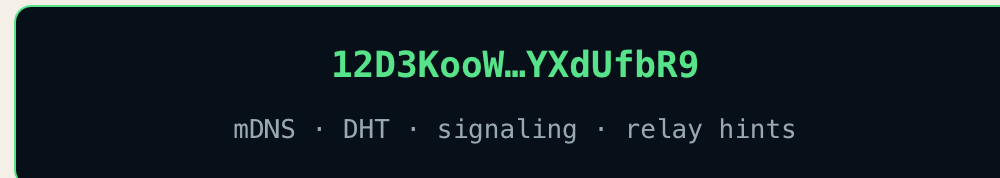
IP меняется – доступ ломается



DNS, проброс порта, VPN-концентратор или ручная инфраструктура.

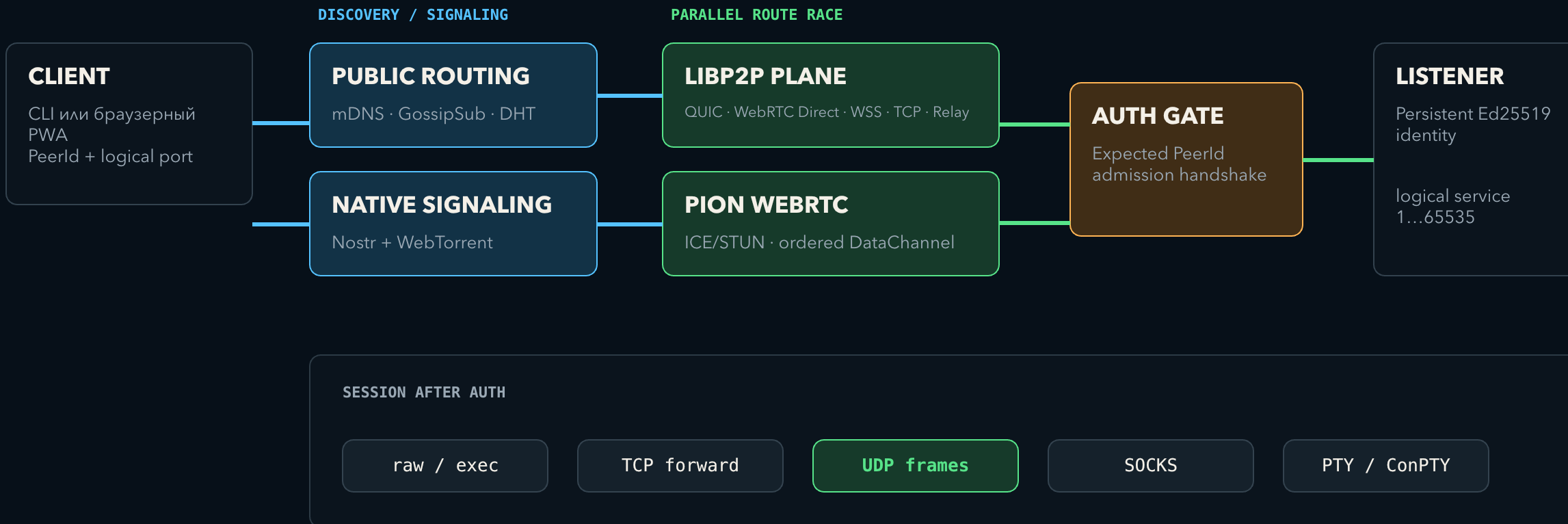
P2P-NETCAT

Identity остаётся стабильной



PeerId выбирает правильного пира; discovery и route race выбирают достижимый путь.

Побеждает первый маршрут, который дошёл до аутентификации.



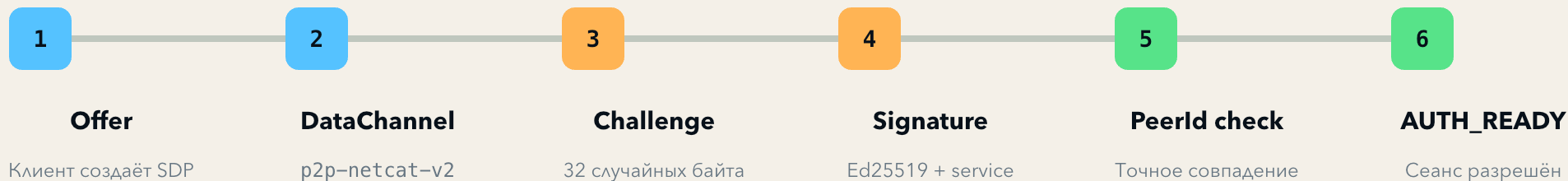
Один CLI покрывает терминал, потоки, datagram и прокси.

РЕЖИМ	ЛОКАЛЬНЫЙ ИНТЕРФЕЙС	УДАЛЁННОЕ ДЕЙСТВИЕ	ЧТО СОХРАНЯЕТСЯ	ФЛАГ
RAW / EXEC	stdin/stdout	поток или команда	байтовый поток	-e
TCP FWD	127.0.0.1:PORT	dial TCP target	соединение	-p
UDP FWD	127.0.0.1:PORT/udp	connected UDP socket	границы пакетов	-u -p
SOCKS	SOCKS4/4a/5	CONNECT от remote	DNS на remote	-S
PTY	интерактивный TTY	Unix PTY / ConPTY	resize + EOF	-i
LISTENER	logical port	много сессий	identity + policy	-l -k

Совместимость wire-протоколов

/p2p-netcat/1.0.0/<port> · /p2p-netcat/udp/1.0.0/<port>

Прикладные байты идут только после доказательства ожидаемого PeerId.



ПОСЛЕ AUTH

**256
КиБ**

сквозное flow window

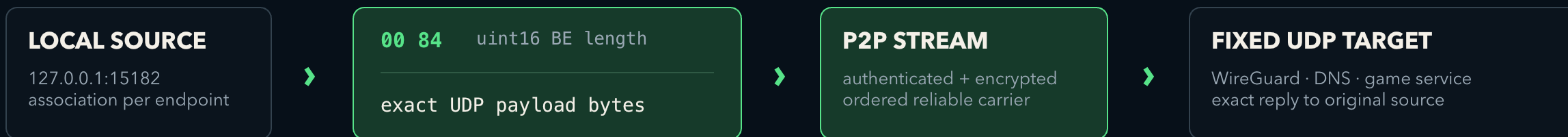
120 сек

reconnect с тем же
логическим stream

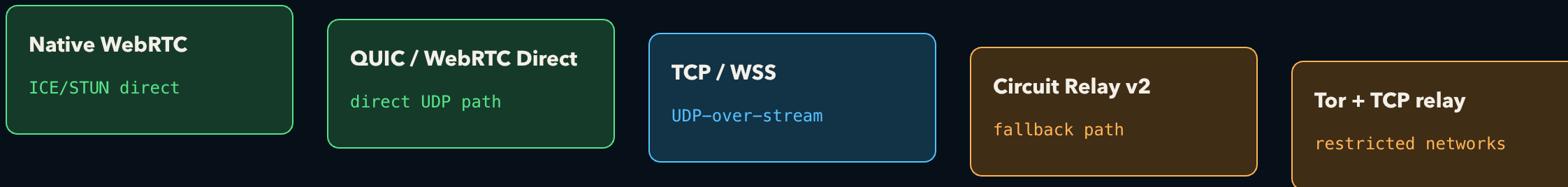
Непроверенный WebRTC-кандидат не может запустить PTY или прикладной поток listener.

Границы UDP-пакетов сохраняются даже поверх TCP/WSS/relay.

DATAGRAM ASSOCIATION

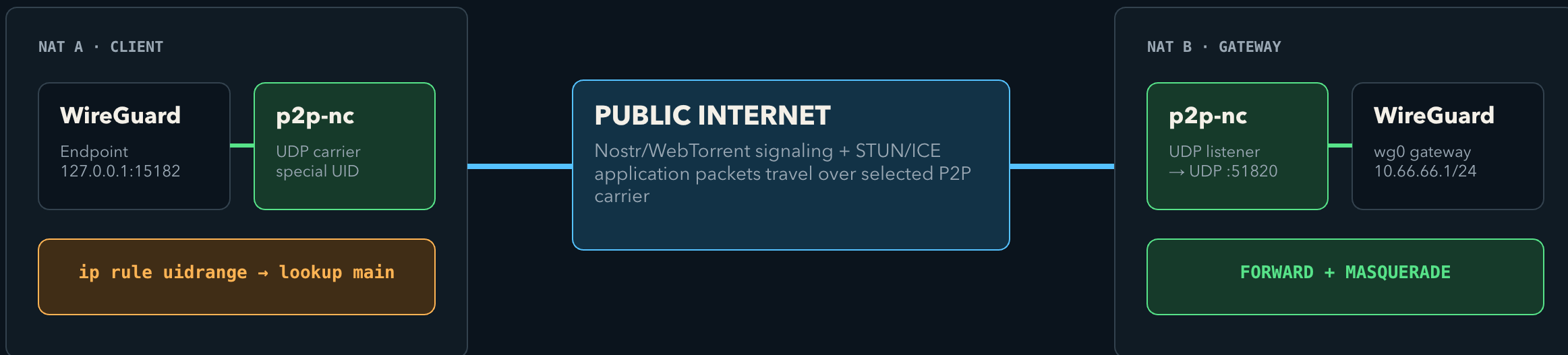


FALLBACK LADDER



Цена fallback: ordered delivery может дать head-of-line blocking.

Carrier остаётся вне default route – поэтому туннель не съедает сам себя.



E2E ПРОВЕРКА

two NAT namespaces



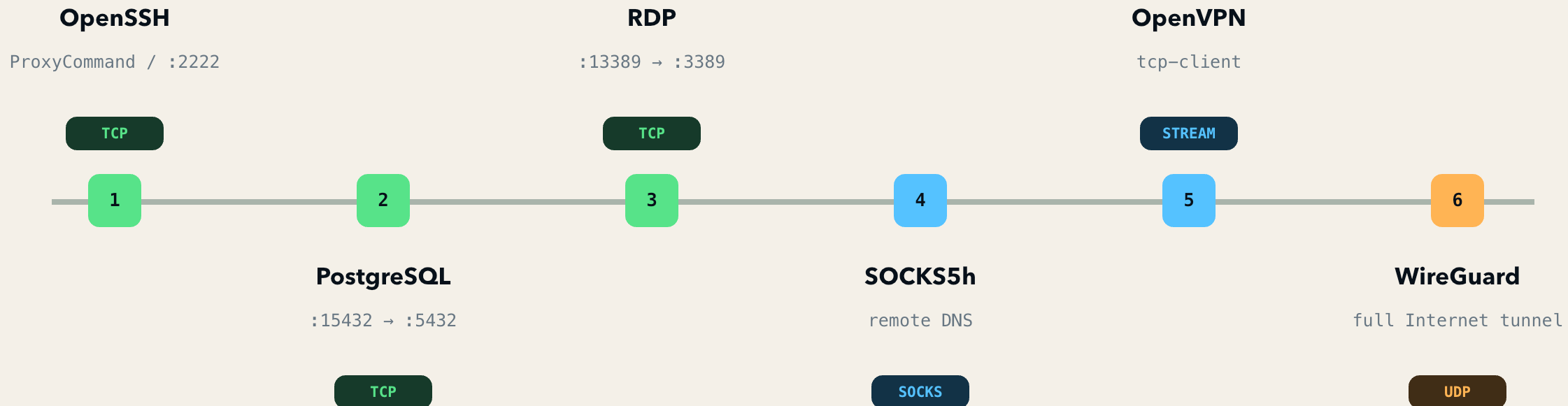
WireGuard handshake



HTTP egress = full-tunnel-ok

Сначала carrier, затем wg-quick up wg0 – policy routing защищает DNS, signaling, STUN и reconnect.

Одна примитивная операция – разные рабочие инструменты.



Logical port выбирает P2P-сервис; он не обязан совпадать с локальным TCP/UDP-портом.

Одна команда установки; четыре поверхности доставки.

VERIFIED DEPLOY

```
curl -fsSL .../deploy.sh | bash
```

- определяет OS + arch
- скачивает release
- проверяет SHA256SUMS
- ставит p2p-nc + pnc

RELEASE BINARIES

Linux · macOS · Windows · Android
amd64 / arm64

GHCR CONTAINER

ghcr.io/santaklouse/go-p2p-netcat
non-root · /config identity

GO INSTALL

cmd/p2p-nc + cmd/pnc
CGO не требуется

BROWSER PWA

Static

GitHub Pages

React UI
Web Worker
IndexedDB routes
Native WebRTC

Тесты бьют по тем границам отказа, которые создаёт архитектура.

NATIVE WEBRTC SOAK

60 / 60

итераций workload

1 152 МиБ

transferredBytes в свежем
JSON-отчёте

Pion DataChannel · SHA-256 · reconnect

CI COVERAGE

Go tests

Linux · macOS · Windows

Race detector

Linux · ./...

Full tunnel

2 NAT + WireGuard + HTTP

Docker

buildx + runtime checks

WebRTC soak

weekly Ubuntu + macOS

ЧТО SOAK НЕ ДОКАЗЫВАЕТ

- доступность публичных Nostr/tracker
- межконтинентальную latency
- все типы NAT и firewall
- фоновые вкладки и сон браузера
- 100% соединений без relay

Три пары команд закрывают самые ценные сценарии.

Во всех примерах используется документационный PeerId и реальные порты.

SSH ЧЕРЕЗ ЛОКАЛЬНЫЙ ПОРТ

```
server$ p2p-nc -l -k -d 127.0.0.1 -p 22 22022
client$ p2p-nc -p 2222 12D3KooWQ3uxpHgjdKE6vGmvzKS8RPbxUDLwJ7XCLaD6YXdUfbR9 22022
client$ ssh -p 2222 alice@127.0.0.1
```

TCP PORT-FORWARDING

```
server$ p2p-nc -l -k -d 127.0.0.1 -p 5432 25432
client$ p2p-nc -p 15432 12D3KooWQ3uxpHgjdKE6vGmvzKS8RPbxUDLwJ7XCLaD6YXdUfbR9 25432
```

WIREGUARD CARRIER ДЛЯ FULL TUNNEL

```
server$ p2p-nc -u -l -k -d 127.0.0.1 -p 51820 35182
client$ sudo wireguard-full-tunnel.sh -- /usr/local/bin/p2p-nc -u --udp-idle-timeout 0 \
      -p 15182 12D3KooWQ3uxpHgjdKE6vGmvzKS8RPbxUDLwJ7XCLaD6YXdUfbR9 35182
client$ sudo wg-quick up wg0
```

Используйте p2p-netcat, когда identity должна жить дольше адреса.

PeerId – стабильная точка входа. Route race и fallback – механизм доставки.

github.com/santaklouse/go-p2p-netcat

ХОРОШО ПОДХОДИТ

- доступ к домашним/edge-хостам
- SSH и закрытые TCP-сервисы
- UDP/WireGuard через NAT
- временные туннели без control plane

НУЖЕН ПЛАН В

- symmetric NAT или UDP blocked
- гарантированный enterprise ingress
- неподдерживаемый SOCKS UDP ASSOCIATE

TURN · Circuit Relay v2 · доступный TCP/WSS route